81. Нулевые указатели

Как и в случае с обычными переменными, **указатели** не инициализируются при создании. Если значение не было присвоено, то указатель по умолчанию будет указывать на любой адрес, содержимым которого является мусор.

Оглавление:

1. Нулевое значение и нулевые указатели
2. Разыменование нулевых указателей
3. Макрос NULL
4. Ключевое слово nullptr в C++11
5. Тип данных std::nullptr_t в C++11

Нулевое значение и нулевые указатели

Помимо адресов памяти, есть еще одно значение, которое указатель может хранить: значение null. **Нулевое значение** (или ***«значение null»***) — это специальное значение, которое означает, что указатель ни на что не указывает. Указатель, содержащий значение null, называется **нулевым указателем**.

В языке C++ мы можем присвоить указателю нулевое значение, инициализируя его/присваивая ему литерал `0`:

```
int *ptr(0);  // ptr теперь нулевой указатель

int *ptr1; // ptr1 не инициализирован
ptr1 = 0; // ptr1 теперь нулевой указатель
```

Поскольку значением нулевого указателя является нуль, то это можно использовать внутри **условного ветвления** для проверки того, является ли указатель нулевым или нет:

```
#include <iostream>

int main()
{
    double *ptr(0);

    if (ptr)
        std::cout << "ptr is pointing to a double value.";
    else
        std::cout << "ptr is a null pointer.";

    return 0;
}
```

> ***Совет:*** Инициализируйте указатели нулевым значением, если не собираетесь присваивать им другие значения.

Разыменование нулевых указателей

Как мы уже знаем из предыдущего урока, разыменование указателей с мусором приведет к неожиданным результатам. С разыменованием нулевого указателя дела обстоят так же. В большинстве случаев вы получите сбой в программе.

В этом есть смысл, ведь разыменование указателя означает, что нужно «перейти к адресу, на который указывает указатель, и достать из этого адреса значение». Нулевой указатель не имеет адреса, поэтому и такой результат.

Макрос NULL

В языке Си (но не в C++) есть специальный макрос препроцессора с именем NULL, который определен как значение `0`. Хоть он и не является частью языка C++, его использование достаточно распространено, и должно работать в каждом компиляторе C++:

```
int *ptr(NULL); // присваиваем адрес 0 указателю ptr
```

Однако, поскольку NULL является макросом препроцессора и, технически, не является частью C++, то его не рекомендуется использовать.

Ключевое слово nullptr в C++11

Обратите внимание, значение `0` не является типом указателя, и присваивание указателю значения `0` для обозначения того, что он является нулевым — немного противоречиво, вам не кажется? В редких случаях, использование `0` в качестве аргумента-литерала может привести к проблемам, так как компилятор не сможет определить, используется ли нулевой указатель или целое число `0`:

```
doAnything(0); // является ли 0 аргументом-значением или аргументом-указателем? (компилятор определит его как целочисленное зна
```

Для решения этой проблемы в C++11 ввели новое **ключевое слово nullptr**, которое также является константой **r-value**.

Начиная с C++11, при работе с нулевыми указателями, использование nullptr является более предпочтительным вариантом, нежели использование `0`:

```
int *ptr = nullptr; // примечание: ptr по-прежнему остается указателем типа int, просто со значением null (0)
```

Язык C++ **неявно преобразует** nullptr в соответствующий тип указателя. Таким образом, в вышеприведенном примере, nullptr неявно преобразуется в указатель типа int, а затем значение `nullptr` присваивается `ptr`.

nullptr также может использоваться для вызова функции (в качестве аргумента-литерала):

```
#include <iostream>

void doAnything(int *ptr)
{
    if (ptr)
        std::cout << "You passed in " << *ptr << '\n';
    else
        std::cout << "You passed in a null pointer\n";
}

int main()
{
    doAnything(nullptr); // теперь аргумент является точно нулевым указателем, а не целочисленным значением

    return 0;
}
```

> ***Совет:*** В C++11 используйте nullptr для инициализации нулевых указателей.

Тип данных std::nullptr_t в C++11

В C++11 добавили новый **тип данных std::nullptr_t**, который находится в **заголовочном файле** cstddef. std::nullptr_t может иметь только одно значение — `nullptr`! Хотя это может показаться немного глупым, но это полезно в одном случае. Если вам нужно написать функцию, которая принимает аргумент nullptr, то какой тип параметра нужно использовать? Правильно! std::nullptr_t. Например:

```
#include <iostream>
#include <cstddef> // для std::nullptr_t

void doAnything(std::nullptr_t ptr)
{
    std::cout << "in doAnything()\n";
}

int main()
{
    doAnything(nullptr); // вызов функции doAnything() с аргументом типа std::nullptr_t

    return 0;
}
```

Вам, вероятно, никогда это не придется использовать, но знать об этом стоит (на всякий пожарный).